Pseudo-outcome weights for deep active sampling

Reference window and the online loop

Each stream is one matrix. The first 280 rows are the source domain and the next 280 are the target, so the reference window has 560 rows. Inside that window the domain indicator is $T = 0$ on the first half and $T = 1$ on the second. The remaining 400 rows are the pool, built so that half of them are target rows, and the 200-row test set is target only. Labels are already observed. The loop decides which of those rows are allowed into SGD. It is not paying for a new label.

A two-layer ReLU network with a linear head, width 64, is warm-started for four epochs on the source half only. That state is copied, so every rule starts from the same weights. Each copy has its own SGD, with learning rate 0.08, momentum 0.9, and weight decay 10^{-4}. The optimizer is not reset between rounds. Gradients are clipped at norm 5. Each of five rounds queries 32 pool rows and then takes three epochs over the cumulative queried set. Round 0 in the figure is the warm start, before any pool row is used. Test accuracy is the fraction of target test rows whose predicted class matches the label.

Doubly robust row weight

The nuisance is fit on the reference window only. μ_0 and μ_1 are ridge regressions of the integer class code, not of a one-hot label. The propensity π is logistic, clipped to [0.01, 0.99]. τ is a ridge regression of the pseudo-outcome ϕ on X, not of the class code. Dimension importances from PermuCATE, a domain forest, and MMD leave-one-covariate-out are computed and stored. They are not the weight plotted here.

For a row (X, Y, T) the doubly robust pseudo-outcome and its squared residual are

$$\phi = (Y - \mu_T)\frac{T - \pi}{\pi(1 - \pi)} + (\mu_1 - \mu_0), \qquad w = (\phi - \tau(X))^2, \tag{1}$$

where $\mu_T = \mu_1$ if $T = 1$ and μ_0 otherwise. Every pool row is scored with $T = 1$, including the source rows in the mixture. A source row whose label sits near μ_0 and away from μ_1 therefore receives a large w. The sampling weight is this per-row residual. A flat comparison uses no weight at all.

How the weight changes a query

The raw residual is replaced by its rank percentile in $(0, 1]$, with ties sharing the average rank. That percentile multiplies a higher-is-better acquisition score. A constant weight is a constant percentile, so the original ranking is unchanged. Uncertainty rules, including the dropout averages and BALD, take the top of score times percentile. Random sampling draws without replacement with probability proportional to the percentile. Core-set and BADGE multiply distance, or the gradient norm, by the same percentile. BIM and DeepFool use $-\|\eta\|^2$, so a smaller flip of the predicted class is queried first, and that score is then multiplied by the percentile. Cluster-margin shortlists ten times the batch by weighted margin, clusters the pool by average linkage into 20 groups, and round-robins from the smallest cluster. BatchBALD is the greedy stand-in: high BALD and low overlap of the mean predictive distribution with rows already chosen. The reference implementation calls `batchbald_redux`, which is not a dependency here. Active learning by learning is a bandit over entropy, margin, and core-set. Its reward is the accuracy of the batch just returned, measured with the network that queried it, divided by the probability of that arm. The update

happens before the SGD step, so the model has not yet fit those labels. k-center uses the same furthest-first rule as core-set and is left out of every mean below.

VAAL, WAAL, learning-loss, the GCN pair, and the semi-supervised trainers are not in the comparison. They replace the network.

Three covariate shifts

The training budget above is shared. Seeds are 0 and 1. The band in the figure is the standard deviation across rules, not across seeds. Pixels and pen coordinates are left on $[0, 1]$. Dividing digit pixels by the source standard deviation blows up blank borders after a roll, so that standardization is not used.

Digits are the sklearn 8×8 images scaled by 16. Target images are rolled two columns to the right and the label is unchanged. Pen digits are the UCI stroke table scaled by 100. The eight horizontal coordinates of a target row, the even columns, are shifted by $+0.25$. Satimage is the OpenML Statlog Landsat table, already standardized to mean zero and unit variance. The first nine bands of a target row are shifted by $+1.25$. The land-cover label is unchanged. In each case the reference is source then target, the pool is mixed half and half, and the test set is target only.

One-pass loader

The three tables are built by the same function. `continuous_loader` yields (x, y, index) in row order and is not shuffled. `one_pass` reads that loader once, takes the reference prefix, and leaves the rest as the pool. The test set is not inside the loader. SGD later uses a second, shuffled loader over only the rows that have been queried. Digits, pen digits, and satimage are not separate training programs. They are three streams into this digestion. The query argument is one dict, `QUERY_STRATEGIES`: each rule once with a flat weight and once with the pseudo-outcome residual.

What the paths show

Figure 1 is the mean test accuracy. Table 1 averages the rules. Table 2 is the same comparison one rule at a time.

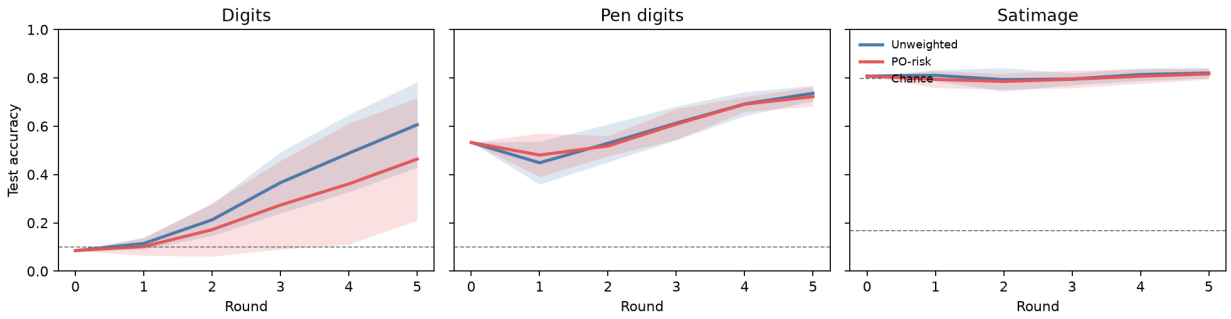


Figure 1: Mean test accuracy by round on the target test set. Round 0 is the source-only warm start. The dashed line is chance, $1/C$. The band is the standard deviation across rules.

Table 1: Mean over query rules and two seeds. k-center is omitted because it repeats core-set. Share is the fraction of queried pool rows that come from the target. Chance is $1/C$.

Dataset	C	Unweighted	PO-risk	AUC plain	AUC PO	Share plain	Share PO
Digits	10	0.606	0.464	0.305	0.237	0.433	0.338
Pen digits	10	0.736	0.722	0.583	0.585	0.502	0.392
Satimage	6	0.820	0.816	0.805	0.798	0.527	0.449

Table 2: Final test accuracy with the pseudo-outcome weight minus the unweighted run, in percentage points. Positive means the weight finished higher on that rule.

Rule	Digits	Pen digits	Satimage
`cluster_margin`	+7.0	+3.5	+0.0
`margin`	+5.5	+7.3	-2.7
`baseline`	-5.5	+6.2	+4.2
`adversarial_bim`	+10.5	-5.5	-1.0
`kmeans`	+0.0	+0.5	+0.0
`margin_dropout`	+2.2	+1.5	-3.5
`adversarial_deepfool`	+1.0	-2.3	-1.5
`albl`	-9.5	+4.0	+2.2
`random`	-10.5	+2.0	-1.2
`batch_bald`	-12.0	-6.0	-3.7
`bald`	-14.5	-5.5	-2.8
`badge`	-16.8	-9.0	+0.2
`entropy`	-29.3	-1.5	+4.7
`coreset`	-29.2	-15.5	+0.7
`least_confidence`	-42.2	-1.0	-1.0
`entropy_dropout`	-43.2	-2.3	-0.5
`least_confidence_dropout`	-55.2	-0.5	-0.7
Mean	-14.2	-1.4	-0.4

Scoring the mixed pool as $T = 1$ is expected to prefer source rows, and it does. The target share of the queried batch falls from 0.433 to 0.338 on digits, from 0.502 to 0.392 on pen digits, and from 0.527 to 0.449 on satimage. The pool rate is 0.50 in every stream.

Target accuracy moves when those source rows are the wrong training data. On digits the unweighted mean ends at 0.606 and the weighted mean at 0.464. The area under the path falls from 0.305 to 0.237. On pen digits the same shift of the batch changes the endpoint only from 0.736 to 0.722. On satimage the source network already scores 0.807 on the target before any pool row is queried, and the two paths finish at 0.820 and 0.816. The residual does what $T = 1$ specifies. It does not raise average target accuracy.

The mean hides a few rules that finish higher. On digits, BIM with the weight ends at 0.853 with target share 0.997, against 0.748 and 0.516 unweighted. Margin is higher with the weight on both digit streams. Least confidence and entropy, with and without dropout, account for most of the loss on digits. Those rule-level gains do not change the average.